

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data into Delta Tables

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a Delta Live Table to load customer and order data from CSV files into delta tables.

Target Cluster Configuration:

- Cluster Name: Gen AI POC Databricks Cluster
- Databricks Runtime Version: 10.4.x-scala2.12
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 Standard_DS3_v2
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: delta-lake, spark-csv

Source Data Details:

- Dataset Name: Customer Data
- Location: ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata``
- Format: CSV
- Description: Customer data in CSV format
- Dataset Name: Order Data
- Location: ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata``
- Format: CSV
- Description: Order data in CSV format

Target Data Details:

- Output Name: Customer Delta Table
- Location: ``catalog="gen_ai_poc_databrickscoe" schema="sdlc_wizard"``
- Format: Delta
- Description: Customer data in delta format
- Output Name: Order Delta Table

- Location: ``catalog="gen_ai_poc_databrickscoe" schema="sdlc_wizard"``
- Format: Delta
- Description: Order data in delta format

Job Flow / Pipeline Stages:

- Read customer CSV data
- Load customer data into delta table
- Read order CSV data
- Load order data into delta table
- Add new column ``TotalAmount`` to order delta table

Data Transformations / Business Logic:

- Step: Load customer data
- Description: Load customer data from CSV into delta table
- Transformation Logic: Use ``spark.read.csv`` to read CSV data and ``write.format("delta")`` to write to delta table
- Step: Load order data
- Description: Load order data from CSV into delta table
- Transformation Logic: Use ``spark.read.csv`` to read CSV data and ``write.format("delta")`` to write to delta table
- Step: Add ``TotalAmount`` column
- Description: Calculate ``TotalAmount`` as product of ``PricePerUnit`` and ``Qty``
- Transformation Logic: Use ``withColumn`` to add new column ``TotalAmount``

Error Handling and Logging:

- Log errors to a designated log file
- Use try-except blocks to catch and handle exceptions

TR-DLT-002: Clean and Transform Customer and Order Data

Related Functional Requirement(s): FR-DLT-002

Objective: Implement data cleaning and transformation logic to remove null and duplicate records from customer and order delta tables.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: Customer Delta Table
- Location: ``catalog="gen_ai_poc_databrickscoe" schema="sdlc_wizard"``
- Format: Delta
- Description: Customer data in delta format

- Dataset Name: Order Delta Table
- Location: `catalog="gen\_ai\_poc\_databricksco" schema="sdlc\_wizard"`
- Format: Delta
- Description: Order data in delta format

Target Data Details:

- Output Name: Clean Customer Delta Table
- Location: `catalog="gen\_ai\_poc\_databricksco" schema="sdlc\_wizard"`
- Format: Delta
- Description: Clean customer data in delta format
- Output Name: Clean Order Delta Table
- Location: `catalog="gen\_ai\_poc\_databricksco" schema="sdlc\_wizard"`
- Format: Delta
- Description: Clean order data in delta format

Job Flow / Pipeline Stages:

- Remove null records from customer delta table
- Remove duplicate records from customer delta table
- Remove null records from order delta table
- Remove duplicate records from order delta table

Data Transformations / Business Logic:

- Step: Remove null records
- Description: Remove null records from customer and order delta tables
- Transformation Logic: Use `filter` to remove null records
- Step: Remove duplicate records
- Description: Remove duplicate records from customer and order delta tables
- Transformation Logic: Use `dropDuplicates` to remove duplicate records

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-003: Create and Load ordersummary Table

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a Delta Live Table to create and load `ordersummary` table with joined customer and order data using SCD type 2.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: Clean Customer Delta Table

- Location: `catalog="gen\_ai\_poc\_databrickscoe" schema="sdlc\_wizard"`
- Format: Delta
- Description: Clean customer data in delta format
- Dataset Name: Clean Order Delta Table
- Location: `catalog="gen\_ai\_poc\_databrickscoe" schema="sdlc\_wizard"`
- Format: Delta
- Description: Clean order data in delta format

Target Data Details:

- Output Name: ordersummary Table
- Location: `catalog="gen\_ai\_poc\_databrickscoe" schema="sdlc\_wizard"`
- Format: Delta
- Description: Joined customer and order data in delta format

Job Flow / Pipeline Stages:

- Create `ordersummary` table if it does not exist
- Join customer and order delta tables on `CustId`
- Load joined data into `ordersummary` table using SCD type 2

Data Transformations / Business Logic:

- Step: Join customer and order data
- Description: Join customer and order delta tables on `CustId`
- Transformation Logic: Use `join` to join customer and order data
- Step: Load data into `ordersummary` table
- Description: Load joined data into `ordersummary` table using SCD type 2
- Transformation Logic: Use `applyChanges` to load data into `ordersummary` table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-004: Update ordersummary Table on Customer Changes

Related Functional Requirement(s): FR-DLT-004

Objective: Implement logic to update `ordersummary` table whenever there is a change in the customer table, maintaining history using SCD type 2.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

- Dataset Name: Customer Delta Table
- Location: `catalog="gen\_ai\_poc\_databrickscoe" schema="sdlc\_wizard"`
- Format: Delta

- Description: Customer data in delta format

Target Data Details:

- Output Name: ordersummary Table
- Location: `catalog="gen\_ai\_poc\_databrickscoe" schema="sdlc\_wizard"`
- Format: Delta
- Description: Updated `ordersummary` table with changed customer data

Job Flow / Pipeline Stages:

- Identify changes in customer delta table
- Update `ordersummary` table by making old records inactive and new records active
- Update `StartDate` and `EndDate` accordingly to maintain history

Data Transformations / Business Logic:

- Step: Identify changes in customer data
- Description: Identify changes in customer delta table
- Transformation Logic: Use `filter` to identify changed records
- Step: Update `ordersummary` table
- Description: Update `ordersummary` table by making old records inactive and new records active
- Transformation Logic: Use `applyChanges` to update `ordersummary` table

Error Handling and Logging: Same as TR-DLT-001**TR-DLT-005: Create and Load customeraggregatespend Table****Related Functional Requirement(s): FR-DLT-005**

Objective: Implement a Delta Live Table to create and load `customeraggregatespend` table with aggregated data from `ordersummary`.

Target Cluster Configuration: Same as TR-DLT-001**Source Data Details:**

- Dataset Name: ordersummary Table
- Location: `catalog="gen\_ai\_poc\_databrickscoe" schema="sdlc\_wizard"`
- Format: Delta
- Description: Joined customer and order data in delta format

Target Data Details:

- Output Name: customeraggregatespend Table
- Location: `catalog="gen\_ai\_poc\_databrickscoe" schema="sdlc\_wizard"`
- Format: Delta
- Description: Aggregated customer spend data in delta format

Job Flow / Pipeline Stages:

- Create `customeraggregatespend` table if it does not exist
- Aggregate `TotalAmount` from `ordersummary` table, grouping by `Name` and `Date`
- Load aggregated data into `customeraggregatespend` table

Data Transformations / Business Logic:

- Step: Aggregate customer spend data
- Description: Aggregate `TotalAmount` from `ordersummary` table, grouping by `Name` and `Date`
- Transformation Logic: Use `groupBy` and `sum` to aggregate customer spend data
- Step: Load data into `customeraggregatespend` table
- Description: Load aggregated data into `customeraggregatespend` table
- Transformation Logic: Use `write.format("delta")` to write to delta table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-006: Implement Delta Live Tables

Related Functional Requirement(s): FR-DLT-006

Objective: Implement the above requirements using Delta Live Tables.

Target Cluster Configuration: Same as TR-DLT-001

Job Flow / Pipeline Stages:

- Implement steps 1 to 5 using Delta Live Tables

Data Transformations / Business Logic: Same as above TRs

Error Handling and Logging: Same as TR-DLT-001